Compiler Assignment 3

PART 1 & 2

The conflict list is
State 52 conflicts: 3 shift/reduce
State 53 conflicts: 3 shift/reduce
State 54 conflicts: 3 shift/reduce
State 55 conflicts: 3 shift/reduce
State 57 conflicts: 20 shift/reduce
State 92 conflicts: 3 shift/reduce
State 93 conflicts: 3 shift/reduce
State 94 conflicts: 3 shift/reduce
State 95 conflicts: 3 shift/reduce
State 96 conflicts: 3 shift/reduce
State 97 conflicts: 3 shift/reduce
State 98 conflicts: 3 shift/reduce
State 99 conflicts: 3 shift/reduce
State 100 conflicts: 3 shift/reduce
State 101 conflicts: 3 shift/reduce
State 102 conflicts: 3 shift/reduce
State 103 conflicts: 3 shift/reduce
State 104 conflicts: 3 shift/reduce
State 105 conflicts: 3 shift/reduce
State 106 conflicts: 3 shift/reduce
State 107 conflicts: 20 shift/reduce
State 108 conflicts: 20 shift/reduce
State 109 conflicts: 20 shift/reduce
State 182 conflicts: 1 shift/reduce

These come mainly from **expression rules**.

Summary of conflicts

The LALR(1) parser generator reported the following conflicts:

State	Conflict Type	Number
52	shift/reduce	3
53	shift/reduce	3
54	shift/reduce	3

55	shift/reduce	3
57	shift/reduce	20
92–97	shift/reduce	3 each
98–109	shift/reduce	20 each
182	shift/reduce	1

Conflict Type 1 — Operator Precedence Ambiguity

Example Item Set (State 93)

expression → expression OR_OP expression •
expression → expression • + expression
expression → expression • - expression
expression → expression • * expression
expression → expression • / expression
expression → expression • % expression
expression → expression • SHL_OP expression
expression → expression • SHR_OP expression
expression → expression • < expression
expression → expression • > expression
expression → expression • LE_OP expression
expression → expression • GE_OP expression
expression → expression • EQ_OP expression
expression → expression • NE_OP expression
expression → expression • AND_OP expression
expression → expression • OR_OP expression
expression → expression • & expression
expression → expression • | expression
expression → expression • ^ expression

Type of conflict

Shift/Reduce conflict

Exact productions involved

expression → expression + expression
expression → expression - expression
expression → expression * expression
expression → expression / expression

expression → expression % expression
expression → expression SHL_OP expression
expression → expression SHR_OP expression
expression → expression < expression
expression → expression > expression
expression → expression LE_OP expression
expression → expression GE_OP expression
expression → expression EQ_OP expression
expression → expression NE_OP expression
expression → expression AND_OP expression
expression → expression OR_OP expression
expression → expression & expression
expression → expression | expression
expression → expression ^ expression

Conflict Type 2 — Unary vs Binary Operator Ambiguity

Example productions involved

expression → * expression
expression → & expression
expression → - expression
expression → expression * expression
expression → expression & expression
expression → expression - expression

Type of conflict

Shift/Reduce conflict

Conflict Type 3 — Dangling Else Ambiguity

Item set (simplified)

if_stmt → IF (expression) statement •
if_stmt → IF (expression) statement • ELSE statement

Type of conflict

Shift/Reduce conflict

Exact productions involved

if_stmt → IF (expression) statement
if_stmt → IF (expression) statement ELSE statement

PART 2

Conflict Type 1 — Operator Precedence Ambiguity

Reason for ambiguity

Original grammar defined expressions as:

```
expression:

    expression '+' expression

  | expression '-' expression

  | expression '*' expression

  | expression '/' expression

  | expression '%' expression

  | expression AND_OP expression

  | expression OR_OP expression

  | expression '<' expression

  | expression '>' expression
```

Grammar modification

Expressions were divided into precedence levels:

```
expression → logical_or_expr

logical_or_expr → logical_or_expr OR_OP logical_and_expr
                | logical_and_expr

logical_and_expr → logical_and_expr AND_OP equality_expr
                 | equality_expr

equality_expr → equality_expr EQ_OP relational_expr
```

```
            | equality_expr NE_OP relational_expr
            | relational_expr

relational_expr → relational_expr '<' additive_expr
              | relational_expr '>' additive_expr
              | additive_expr

additive_expr → additive_expr '+' multiplicative_expr
            | additive_expr '-' multiplicative_expr
            | multiplicative_expr

multiplicative_expr → multiplicative_expr '*' unary_expr
                  | multiplicative_expr '/' unary_expr
                  | unary_expr
```

Now parser always evaluates, * before + and && before || and likewise

Example 1 — Operator Precedence Fix

Input

a + b * c;

Before grammar fix (ambiguous)

Two possible parses:

Parse 1

```
    *
   / \
  +   c
 / \
 a   b
```

meaning:

(a + b) * c

Parse 2

```
    +
   / \
  a   *
     / \
    b   c
```

meaning:

a + (b * c)

Parser could not decide which rule to apply first.

After grammar fix

Operator precedence defined:

* has higher precedence than +

Correct parse:

```
    +
   / \
  a   *
     / \
    b   c
```

Final interpretation:

a + (b * c)

Conflict removed.

Conflict Type 2 — Unary vs Binary Operator Ambiguity

Reason for ambiguity

Operators like `*`, `&`, and `-` can be used as both:

- unary operator
- binary operator

Example ambiguous expression:

*x * y
-a + b

Parser cannot determine operator grouping.

Problem in original grammar

Unary and binary operators were mixed:

```
expression:
    '*' expression
  | '&' expression
  | '-' expression
  | expression '*' expression
  | expression '&' expression
```

Example ambiguous input:

```
*x * y
```

Parser confusion:

```
(*x) * y
*(x * y)
```

Grammar modification

Unary expressions separated:

```
unary_expr:
    '-' unary_expr
  | '!' unary_expr
  | '~' unary_expr
  | '*' unary_expr
  | '&' unary_expr
  | primary_expr
```

Binary expressions handled separately:

```
multiplicative_expr: multiplicative_expr '*' unary_expr
```

Example 2 — Unary vs Binary Operator Fix

Input

```
*x * y;
```

Before fix (ambiguous)

Possible interpretations:

(*x) * y

OR

*(x * y)

Parser unsure whether first * is unary or binary.

After fix

Unary operators separated in grammar.

Correct parse:

```
  *

 / \

*   y

|

x
```

Final interpretation:

(*x) * y

Conflict removed.

Conflict Type 3 — Dangling Else Ambiguity

Reason for ambiguity

Example:

```
if(a)
  if(b)
    x = 1;
  else
    x = 2;
```

Parser cannot determine whether `else` belongs to:

inner if
OR
outer if

Problem in original grammar

```
statement:
      var_decl
    | assignment ';'
    | expression ';'
    | block
    | if_stmt
    | loop_stmt
    | switch_stmt
    | BREAK ';'
    | CONTINUE ';'
    | RETURN expression ';'
    ;


if_stmt:
     IF '(' expression ')' statement
   | IF '(' expression ')' statement ELSE statement
```

Ambiguous input:

```
if(a)
  if(b)
     x=1;
  else
     x=2;
```

Two interpretations possible:

1. else belongs to inner if
2. else belongs to outer if

Conflict occurred in state:

Grammar modification

We add these below other precedence rules:
%nonassoc IFX
%nonassoc ELSE

/* dangling else resolved */

```
statement:
      var_decl
    | assignment ';'
    | expression ';'
    | block
    | loop_stmt
    | switch_stmt
    | BREAK ';'
    | CONTINUE ';'
    | RETURN expression ';'
    | IF '(' expression ')' statement %prec IFX
    | IF '(' expression ')' statement ELSE statement
    ;
```

Why this resolves conflict

The rule:

IF '(' expression ')' statement %prec IFX

assigns lower precedence to IF without ELSE.

Parser prefers:

IF '(' expression ')' statement ELSE statement

So ELSE always attaches to the nearest IF.

Example 3 — Dangling Else Fix

Input

```
if(a)
  if(b)
     x=1;
```

```
  else
     x=2;
```

Before fix (ambiguous)

ELSE could match:

Outer IF:

```
if(a)
{
  if(b)
     x=1;
}
else
{
  x=2;
}
```

OR inner IF:

```
if(a)
{
  if(b)
     x=1;
  else
     x=2;
}
```

Parser ambiguity caused shift/reduce conflict.

After fix

Grammar ensures ELSE attaches to nearest IF.

Correct parse:

```
if(a)
{
  if(b)
     x=1;
  else
     x=2;
}
```

Conflict removed.

Part 3: Reverse Derivation Tree

```
func main() {
    var x = 5;
    if (x == 0) {
        var y = x + 1;
    }
}
var x = 5 * 3 + 2;
```

For this code we get the following derivation tree elements in the sequence.

```
ayush@Honormagicbook:/mnt/c/Users/ayush/OneDrive/Desktop/CScript$ ./a.out tests/parse_errors.cs

Parsing Successful

Reverse Derivation Tree:
int
var_init=
var_decl
id
int
==
id
int
+
var_init=
var_decl
block
if
stmt_list
block
func_decl
int
int
*
int
+
var_init=
var_decl
global_list
program
```

These tree nodes can be mapped like this for understanding

Group 1 — `var x = 5;` inside `func main()`

- `int` ← literal 5 (right-hand side of assignment)
- `var_init=` ← IDENTIFIER '=' expression → var_init (x = 5)
- `var_decl` ← VAR var_init_list ';' → var_decl

This is the first statement inside `func main()`. The integer literal `5` is reduced first, then it becomes part of `var_init=` (an initialized variable), which is then wrapped into `var_decl`.

Group 2 — condition `x == 0` of the `if` statement

- `id` ← identifier x (left operand)

- `int` ← literal 0 (right operand)
- `==` ← x == 0 reduced to equality_expr

The condition of the `if` statement is reduced here. First `x` is reduced to an `id` node, then `0` is reduced to an `int` node, and finally the `==` operator combines them into an equality expression.

Group 3 — `var y = x + 1;` inside the `if` block

- `id` ← identifier x (left operand of +)
- `int` ← literal 1 (right operand of +)
- `+` ← x + 1 reduced to additive_expr
- `var_init=` ← y = x + 1 reduced to var_init
- `var_decl` ← VAR var_init_list ';' → var_decl

This is the statement inside the `if` block body. The expression `x + 1` is reduced bottom-up — `x` and `1` first, then `+` combines them. The result becomes the initializer of `var y`, forming `var_init=`, which is then wrapped in `var_decl`.

Group 4 — structure of `if` and `func main()`

- `block` ← '{' var_y_decl '}' → inner if block
- `if` ← IF '(' x == 0 ')' block → if_stmt
- `stmt_list` ← var_x_decl + if_stmt → statement_list
- `block` ← '{' stmt_list '}' → function body block
- `func_decl` ← FUNC main '(' ')' block → func_decl

After all inner expressions are reduced, the structural nodes are built. The inner `if` block is reduced first, then the `if` statement itself is formed using the condition (`x == 0`) and the block. The statement list combines `var x = 5` and the `if` statement. Finally, the entire function body block and `func_decl` are formed.

Group 5 — global `var x = 5 * 3 + 2;`

- `int` ← literal 5
- `int` ← literal 3
- `*` ← 5 * 3 reduced to multiplicative_expr
- `int` ← literal 2
- `+` ← (5 * 3) + 2 reduced to additive_expr
- `var_init=` ← x = 5 * 3 + 2 reduced to var_init
- `var_decl` ← VAR var_init_list ';' → var_decl

This is the global variable declaration. The expression `5 * 3 + 2` is reduced according to operator precedence — multiplication before addition. So `5 * 3` is reduced first, then adding `2` gives the final expression. This is then wrapped into `var_init=a` and `var_decl`.

Group 6 — root nodes

- `global_list` ← func_decl + var_decl combined into global_list
- `program` ← global_list reduced to program (root)

Part 4: Parsing Table Output

The LALR(1) parsing table is automatically generated during the build process and written to `parsing_table.txt`. The table is derived from `y.output`. We have displayed the parsing table and it is saved in a separate file called parsing_table.txt. Once we run the commands 'make clean' and 'make' the file is created. The parsing table is implemented in `print_table.c` and is invoked as a separate build step in the `Makefile`

Part 5: Error Diagnostics

Errors are classified based on the unexpected token encountered during parsing. Different categories of errors were identified and specific diagnostic messages were provided for each category. We report the first parsing error reported to the user.

For example

1)

```
var x = 10;
var y = 10;
var z = x + y
```

```
ayush@Honormagicbook:/mnt/c/Users/ayush/OneDrive/Desktop/CScript$ ./a.out tests/parse_errors.cs

╔═══════════════════════════╗
╚           PARSE ERROR    ╝

 Location : line 3, column 8
 Token    : 'var' (unexpected keyword)
 Error    : missing ';' at end of line 3
 Fix      : add ';' at the end of line 3
═══════════════════════════
```

2)

```
var x = 10;
var  = 10;
var z = x + y
```

```
ayush@Honormagicbook:/mnt/c/Users/ayush/OneDrive/Desktop/CScript$ ./a.out tests/parse_errors.cs

╔═══════════════════════════════════╗
╚═══ PARSE ERROR                   ═╝

  Location : line 2, column 5
  Token    : '='
  Error    : unexpected token '='
  Fix      : check syntax around this location
═══════════════════════════════════════
```

3)

```
var x = 5;
// if (x > 0) {
//     var y = x + 1;
// }

var x = 5;
var y = x + ;
var z = * 3;
```

```
╔═══════════════════════════════╗
╚═══ PARSE ERROR               ═╝

  Location : line 6, column 9
  Token    : ';'
  Error    : unexpected ';' — missing expression?
  Fix      : check expression before ';'
  Example  : var x = a + b;  <-- expression needed
═══════════════════════════════
```

4) Successful execution without any parsing errors

```
func main() {
    var x = 5;
    if (x == 0) {
        var y = x + 1;
    }
}
var x = 5 * 3 + 2;
// var y = x + ;
// var z = * 3;
```

```
ayush@Honormagicbook:/mnt/c/Users/ayush/OneDrive/Desktop/CScript$ ./a.out tests/parse_errors.cs

Parsing Successful

Reverse Derivation Tree:
int
var_init=
var_decl
id
int
==
id
int
+
var_init=
var_decl
block
if
stmt_list
block
func_decl
int
int
*
int
+
var_init=
var_decl
global_list
program
```

5)

```
// var y = x + ;


var x = 5;
if (x == 0) {
    var y = x + 1;
}


var x = 5 * 3 + 2;
// var y = x + ;
// var z = * 3;
```

```
ayush@Honormagicbook:/mnt/c/Users/ayush/OneDrive/Desktop/CScript$ ./a.out tests/parse_errors.cs

╔═══════════════════════════════╗
╚           PARSE ERROR         ╝

  Location : line 3
  Token    : 'if'
  Error    : 'if' cannot appear at global scope
  Fix      : control flow statements must be inside a func
  Example  :
             func main() {
                 if (x > 0) { ... }
             }
═══════════════════════════════
```

Part 6: Additional Information

1. Grammar Hierarchy and Language Design

The language **CScript** is designed as a statically-structured, C-like language. The grammar is deliberately stratified into clear levels of precedence to avoid ambiguity:

```
program
└── global_list
    └── global
        ├── var_decl
        ├── func_decl
        ├── struct_decl
        └── extern_decl
            └── block
                └── statement_list
                    └── statement
                        └── expression
                            ├── logical_or_expr
                            ├── logical_and_expr
                            ├── bitwise_or_expr
                            ├── bitwise_xor_expr
                            ├── bitwise_and_expr
                            ├── equality_expr
                            ├── relational_expr
                            ├── shift_expr
                            ├── additive_expr
                            ├── multiplicative_expr
                            ├── unary_expr
                            ├── postfix_expr
                            └── primary_expr
```

This 12-level expression hierarchy completely eliminates all operator precedence conflicts that would otherwise arise from a flat `expression → expression OP expression` grammar.

2. Conflict Analysis and Resolution

The original flat grammar (before the expression hierarchy was introduced) produced 182 shift/reduce conflicts across 24 states. These were traced to three root causes:

Cause	Conflicts	Resolution
Missing `%left`/`%right` for `SHL_OP`, `SHR_OP`, `` ` ``	`,` `^`, `&` `` ` ``	100
No `%prec` on unary operators `-`, `*`, `!`, `~`, `&`	81	Separate `unary_expr` rule

Dangling else ambiguity	1	`%nonassoc IFX` / `%nonassoc ELSE`

After applying the stratified expression grammar and the dangling-else fix, the final grammar produces zero conflicts.